

UNIT 3

UNIT 3 — Introduction to the Relational Model

1. The Relational Data Model

The relational database model is the most widely used database model in industry today. It was formally proposed by **E.F. Codd** in 1970. The model is based on the mathematical concept of a **relation** (essentially a table).

In a relational database:

- Data is organized into **tables** (called relations)
- Each table represents a specific entity type or relationship
- All data interactions (queries, updates) are done through a structured query language (SQL)
- Integrity constraints ensure data accuracy and consistency

The relational model allows for the definition of data structures, storage, retrieval operations, and integrity constraints, all in a clean and mathematically well-defined way.

2. Structure of Relational Databases — Key Terminology

Term	Definition
Relation	A table consisting of rows and columns. Represents an entity set or a relationship.
Tuple	A single row in a table. Represents one record/instance of the entity.
Attribute	A column in a table. Represents a property or characteristic of the entity.
Domain	The set of all permitted values for a given attribute.
Degree	The number of attributes (columns) in a relation.
Cardinality	The number of tuples (rows) in a relation at a given point in time.
Relation Schema	The logical structure of a relation — its name, attribute names, and their domains. Example: <code>Student(Roll_No, Name, Branch, Year)</code>
Relation Instance	The actual set of tuples stored in a relation at a particular moment in time.
NULL	A special marker indicating that the value is unknown, missing, or not applicable.

Illustrative Example:

Relation Schema: Student(Roll_No, Name, Branch, Year)

Relation Instance (current data):

Roll_No	Name	Branch	Year
101	Alice	CS	2
102	Bob	IT	3
103	Carol	CS	1

- Degree = 4, Cardinality = 3
-

3. Properties of a Relation

A relation (table) in the relational model must satisfy the following properties:

1. **Atomicity of values:** Every cell in the table contains a single, indivisible (atomic) value. No lists, sets, or nested tables are allowed inside a cell.
 2. **Unique attribute names:** Every column in a table must have a distinct name.
 3. **Homogeneous columns:** All values within a single column must belong to the same domain (the same data type).
 4. **Uniqueness of tuples:** No two rows in a table can be identical. Every tuple must be unique.
 5. **No ordering of rows:** The order in which tuples appear in a relation does not matter; a relation is a set of tuples.
 6. **No ordering of columns:** The order of columns is also not significant; columns are identified by their names, not positions.
-

4. Database Schema and Instance

A **Database Schema** is the overall logical structure of the entire database — it describes all the tables, their attributes, and the constraints between them. The schema is defined once during database creation and rarely changes.

A **Database Instance** is the actual data stored in the database at a specific moment in time. As data is inserted, updated, or deleted, the instance changes, but the schema generally remains the same.

Analogy: Think of a schema as a class definition in programming (the structure/blueprint), and an instance as the actual object created from it (the data at runtime).

5. Types of Keys

Keys are attributes (or combinations of attributes) that are used to uniquely identify tuples in a relation and to establish relationships between relations.

5.1 Super Key

A **super key** is any set of one or more attributes whose combined values can uniquely identify every tuple in a relation. A super key may contain extra (redundant) attributes that are not necessary for uniqueness.

Example: In `Customer(Customer_ID, Name, Address)`:

- `{Customer_ID}` is a super key
- `{Customer_ID, Name}` is also a super key (even though Name alone is not unique, adding it doesn't break uniqueness)
- `{Name, Address}` may also be a super key if the combination is unique

5.2 Candidate Key

A **candidate key** is a **minimal super key** — a super key from which no attribute can be removed while still maintaining uniqueness. In other words, every attribute in a candidate key is necessary for uniqueness.

There can be multiple candidate keys for a relation.

Example:

Customer_ID	Name	Address
1	John	123 Road, London
2	John	A-21, Chicago
3	Lara	A-21, Chicago

- `{Customer_ID}` → unique, minimal → candidate key
- `{Name, Address}` → unique combination, neither alone is unique → candidate key
- `{Customer_ID, Name}` → unique but not minimal (Customer_ID alone works) → super key, NOT a candidate key

5.3 Primary Key

The **primary key** is the one candidate key that is chosen by the database designer to officially identify each tuple. It must satisfy:

- **Uniqueness:** No two tuples can have the same primary key value
- **Non-null:** The primary key attribute can never be NULL

Only **one primary key** is allowed per relation.

5.4 Alternate Key

The candidate keys that are **not chosen** as the primary key are called **alternate keys**. They are valid identifiers but are not used as the official identifier.

Example: If `Roll_No` is selected as the primary key of a Student relation, then `Aadhaar_No` and `Email` (if they were candidate keys) become alternate keys.

5.5 Foreign Key

A **foreign key** is an attribute (or set of attributes) in one relation that refers to the **primary key** of another relation (or possibly the same relation). It is the mechanism through which relationships between tables are enforced in the relational model.

Terminology:

- **Child table (Referencing table):** The table that contains the foreign key
- **Parent table (Referenced table):** The table whose primary key is being referenced

Rule: Every non-null foreign key value must match an existing primary key value in the parent table. This is the **referential integrity constraint**.

Example:

```
Department(Dept_ID, Dept_Name, Location)    ← Parent table
Student(Roll_No, Name, Dept_ID)              ← Child table
                                         ↑ Foreign Key referencing Department(Dept_ID)
```

If a student's `Dept_ID = 10`, then `Dept_ID = 10` must exist in the Department table. A student cannot belong to a department that doesn't exist.

Keys Summary Table

Key Type	Description
Super Key	Any attribute set that uniquely identifies a tuple (may have extras)
Candidate Key	Minimal super key — no attribute can be removed
Primary Key	The chosen candidate key; must be unique and NOT NULL
Alternate Key	Candidate keys not selected as the primary key
Foreign Key	References the primary key of another (or same) table

6. Relational Integrity Constraints

Integrity constraints are rules that ensure the accuracy, consistency, and validity of data stored in the database. Every relation must satisfy these conditions at all times.

6.1 Domain Constraints

Every attribute value must belong to the **defined domain** (permitted value range) for that attribute. The domain constraint ensures that the data stored in a column is of the correct type and falls within the acceptable range.

Examples:

- `Age` must be a positive integer (cannot be negative or fractional)
- `Gender` must be one of `'M'`, `'F'`, or `'Other'`
- `Telephone number` digits must be in the range 0–9
- `Month` must be a value between 1 and 12

6.2 Key Constraints (Entity Integrity)

Key constraints state:

- No two tuples in a relation can have the same value for the primary key attribute(s)
- The primary key attribute can **never** contain a NULL value

These constraints ensure that every entity in the relation is uniquely identifiable.

6.3 Referential Integrity Constraints

Referential integrity ensures consistency between related tables. It states:

If a foreign key value exists in the referencing (child) table, the same value must exist as a primary key in the referenced (parent) table.

A foreign key value can be NULL (indicating no relationship is defined for that tuple), but if it is not NULL, it must match a valid primary key in the parent table. `If not, the record not matching the primary key is called an orphaned record.`

Example violation: If a student's `Dept_ID = 99` but no department with `Dept_ID = 99` exists in the Department table, the referential integrity constraint is violated.

7. Relational Algebra

Relational algebra is a **procedural query language** that forms the theoretical foundation of relational databases. It takes one or two **relations as input** and produces a **new relation as output**. SQL, the practical language used to interact with databases, is largely based on the operations defined in relational algebra.

The key characteristic of relational algebra is that it is **procedural** — it tells the database system exactly *how* to retrieve the required data, step by step. Operations can be composed together, meaning the output of one operation can be the input of the next.

Operators in relational algebra are of two kinds:

- **Unary operators** — take a single relation as input (Select, Project, Rename)
 - **Binary operators** — take two relations as input (Union, Set Difference, Cartesian Product, Join)
-

8. Basic Operators of Relational Algebra

8.1 Selection — σ (Sigma)

The selection operation **filters rows** from a relation based on a given condition (predicate). It returns only those tuples that satisfy the predicate. All attributes of the selected tuples are retained.

Notation: `$\sigma_{condition}(Relation)$`

The condition can use:

- Comparison operators: `=`, `≠`, `<`, `>`, `≤`, `≥`
- Logical connectives: `AND`, `OR`, `NOT`

Examples:

Select all books where the subject is "database":

```
 $\sigma$  subject = 'database' (Books)
```

Select books where the subject is "database" AND the price is 450:

```
 $\sigma$  subject = 'database' AND price = 450 (Books)
```

Select books where subject is "database" AND price is 450, OR books published after 2010:

```
 $\sigma$  subject = 'database' AND price = 450 OR year > 2010 (Books)
```

Selection is a **horizontal** operation — it produces a subset of rows from the original table, without changing the columns.

8.2 Projection — Π (Pi)

The projection operation **filters columns** from a relation. It returns only the specified attributes for all tuples. Since a relation cannot have duplicate tuples, any duplicate rows in the result are automatically eliminated.

Notation: `$\Pi attr1, attr2, \dots (Relation)$`

Examples:

Get only the subject and author columns from Books:

Π subject, author (Books)

Get only the names of all students:

Π Name (Student)

Projection is a **vertical** operation — it produces a subset of columns from the original table.

8.3 Union — $\cup$

The union operation returns all tuples that appear in **either or both** of two relations. Duplicate tuples are automatically eliminated from the result.

Notation: $R \cup S$

Conditions for Union (Union Compatibility):

Both relations must be **union-compatible**, meaning:

- They must have the **same number of attributes** (same degree)
- The corresponding attributes must have **compatible domains**

Example:

Get names of all authors who have written either a book or an article (or both):

Π author (Books) $\cup$ Π author (Articles)

8.4 Set Difference — $-$

The set difference operation returns tuples that are present in the **first relation but not in the second**. Union compatibility is required here too.

Notation: $R - S$

Example:

Get names of authors who have written books but have NOT written any articles:

Π author (Books) $-$ Π author (Articles)

8.5 Cartesian Product — $\times$ (Cross Product)

The Cartesian product combines **every tuple of relation R with every tuple of relation S**, producing all possible pairs.

Notation: $R \times S$

If R has m tuples and p attributes, and S has n tuples and q attributes:

- Result has $m \times n$ tuples
- Result has $p + q$ attributes

Cartesian product by itself is rarely useful — it generates a huge number of often-meaningless combinations. Its real power comes when combined with selection (which gives us a Join operation).

Example:

```
 $\sigma_{\text{author} = \text{'nishant'}} (\text{Books} \times \text{Articles})$ 
```

This yields all books and articles written by nishant, by first forming all combinations and then filtering.

8.6 Rename — ρ (Rho)

The rename operation allows us to **give a name** to the result of a relational algebra expression, or to rename an existing relation. This is useful when writing complex expressions that refer to the result of a sub-expression multiple times, or when doing self-joins.

Notation: $\rho_x(E)$

This saves the result of expression E under the name x.

Example:

Rename the Employee relation to Emp:

```
 $\rho_{\text{Emp}} (\text{Employee})$ 
```

9. Natural Join — $\bowtie$

The natural join is a derived operation. It combines two relations based on **all attributes that have the same name** in both relations. Tuples are matched where the values of those common attributes are equal. In the result, the duplicate common attribute columns are merged into one.

Notation: $R \bowtie S$

Example:

If `Student(Roll_No, Name, Dept_ID)` and `Department(Dept_ID, Dept_Name)` share the attribute `Dept_ID`, then:

```
Student  $\bowtie$  Department
```


produces a table with columns `Roll_No, Name, Dept_ID, Dept_Name`, where each student row is combined with the matching department row.
